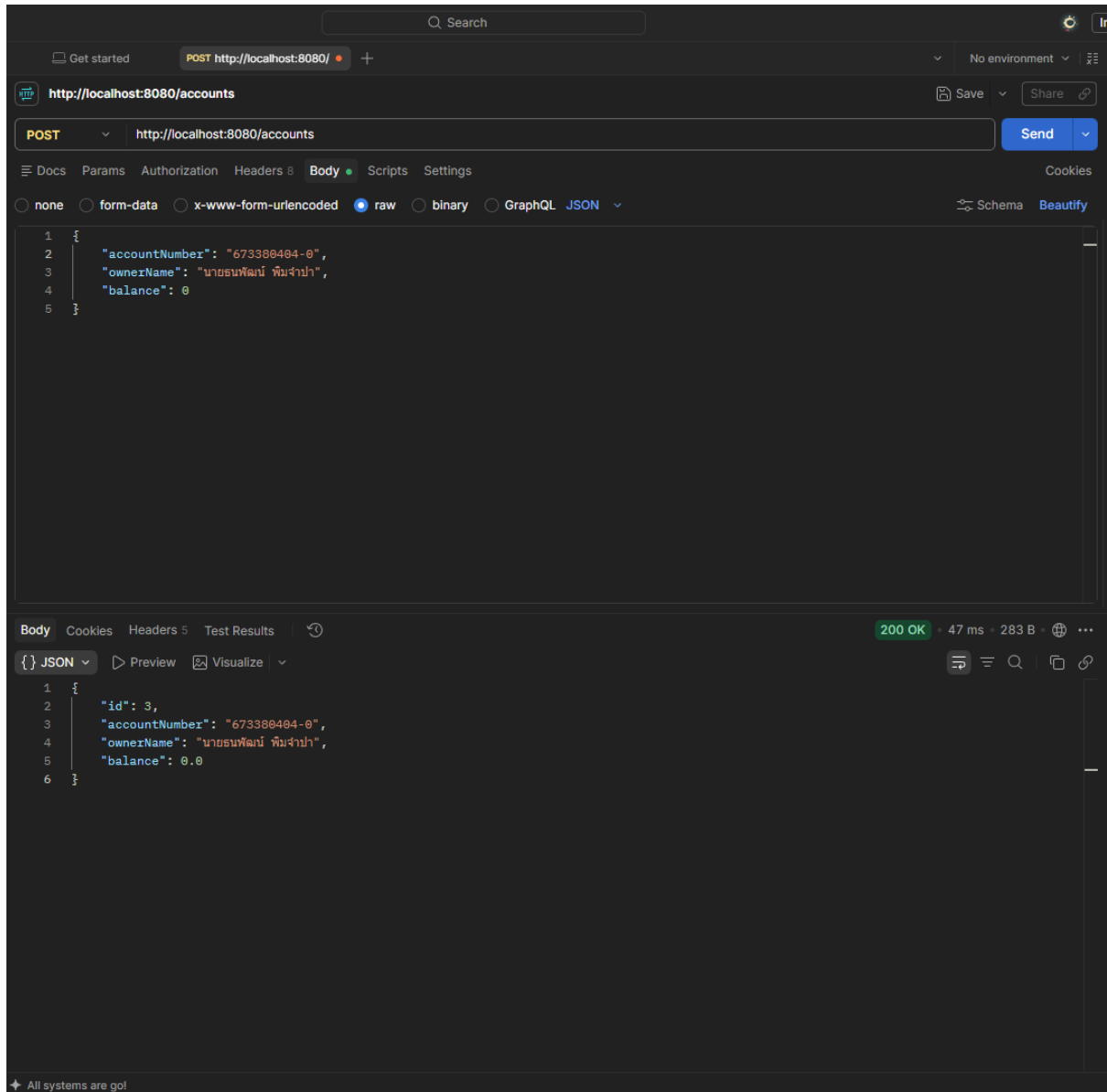


นายธนพัฒน์ พิมพ์จำปา 673380404-0 sec04

1. การสร้าง Account

- รูปภาพ

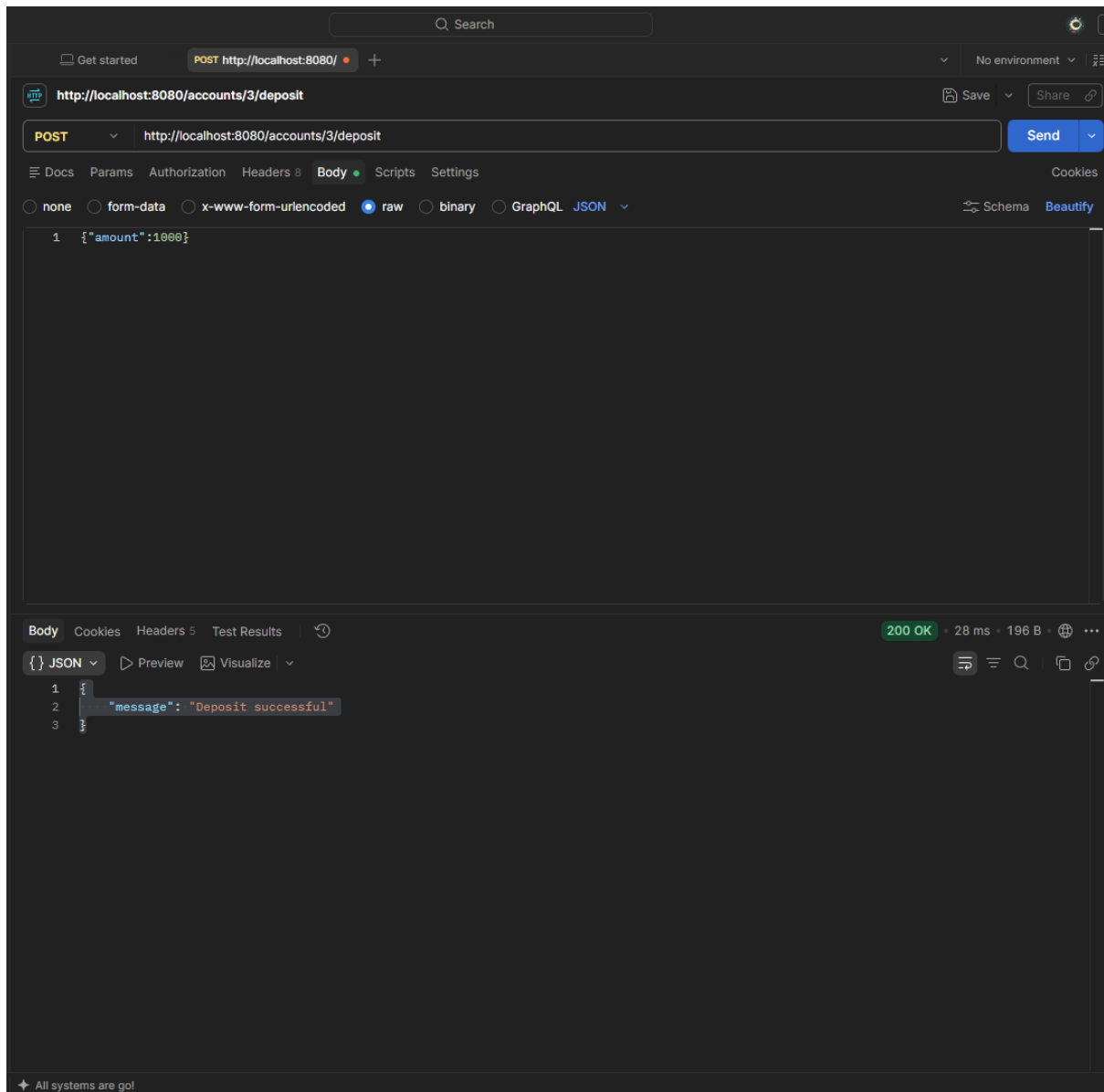


- อธิบายผล

ส่ง POST ไปที่ `/accounts` Controller เรียก `AccountService.create()` บันทึกบัญชีใหม่ลงตาราง `account` ระบบตอบกลับ 200 OK พร้อม `id` ที่ฐานข้อมูลสร้างให้อัตโนมัติ (`balance` เริ่มต้น = 0)

2. การฝากเงินสำเร็จ

- รูปภาพ

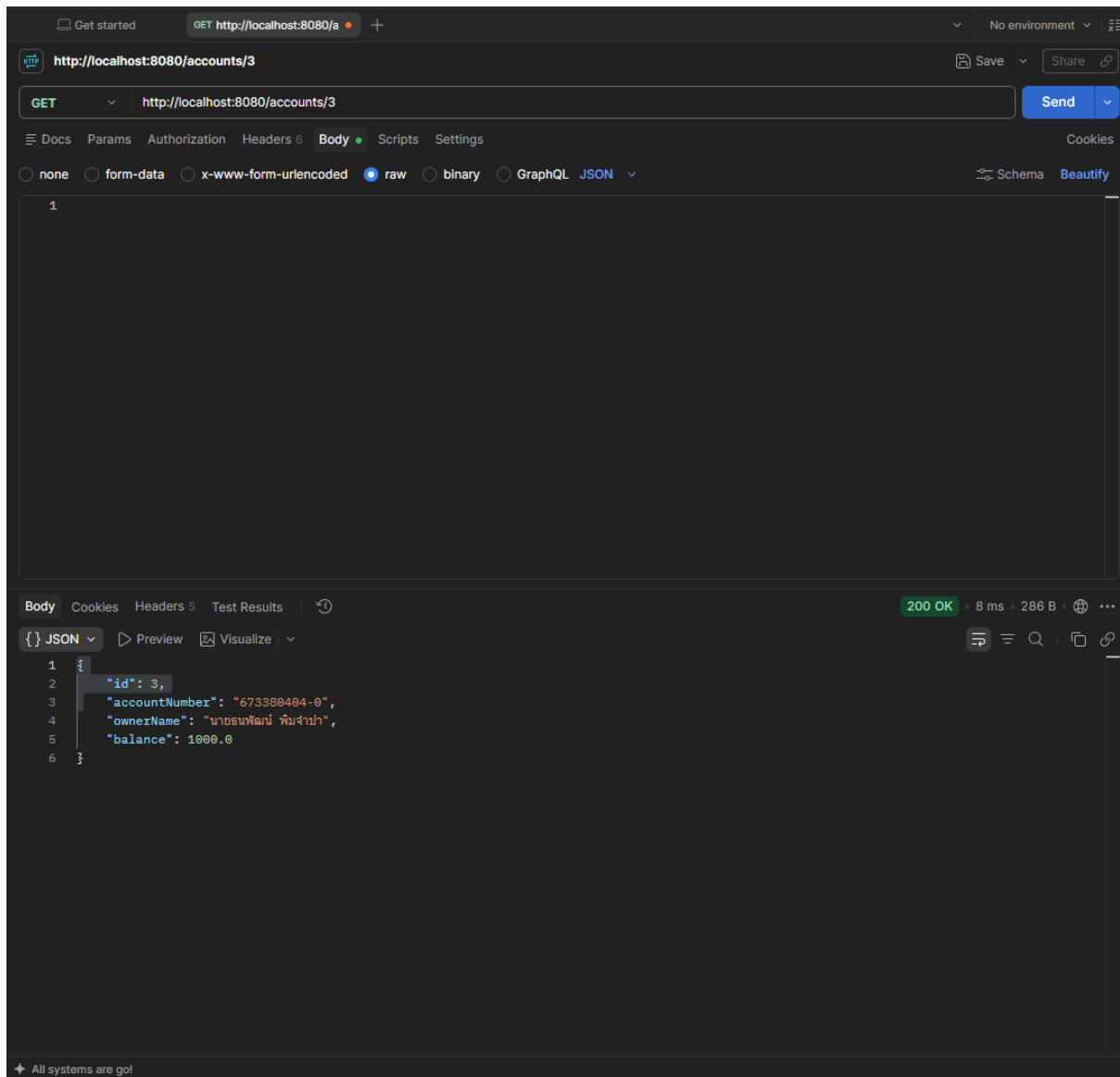


- อธิบายผล

ส่งคำขอ POST /accounts/1/deposit พร้อมจำนวนเงิน 1000 บาท Controller เรียกใช้ DepositService.deposit() ซึ่งมี @Transactional กำกับอยู่ เมธอดนี้ทำงาน 2 ขั้นตอนภายในทรานแซกชันเดียวกัน คือ (1) เพิ่มยอด balance ของบัญชีขึ้น 1000 แล้วบันทึก และ (2) สร้างรายการใหม่ลงตาราง deposit_transaction เพื่อเก็บประวัติการฝาก เมื่อทั้งสองขั้นตอนสำเร็จ ระบบจึง COMMIT ยืนยันข้อมูลลงฐานข้อมูลพร้อมกัน แล้วตอบกลับสถานะ 200 OK พร้อมข้อความ {"message": "Deposit successful"} ยืนยันว่าการฝากเงินเสร็จสมบูรณ์

3. ตรวจสอบ Account

- รูปภาพ

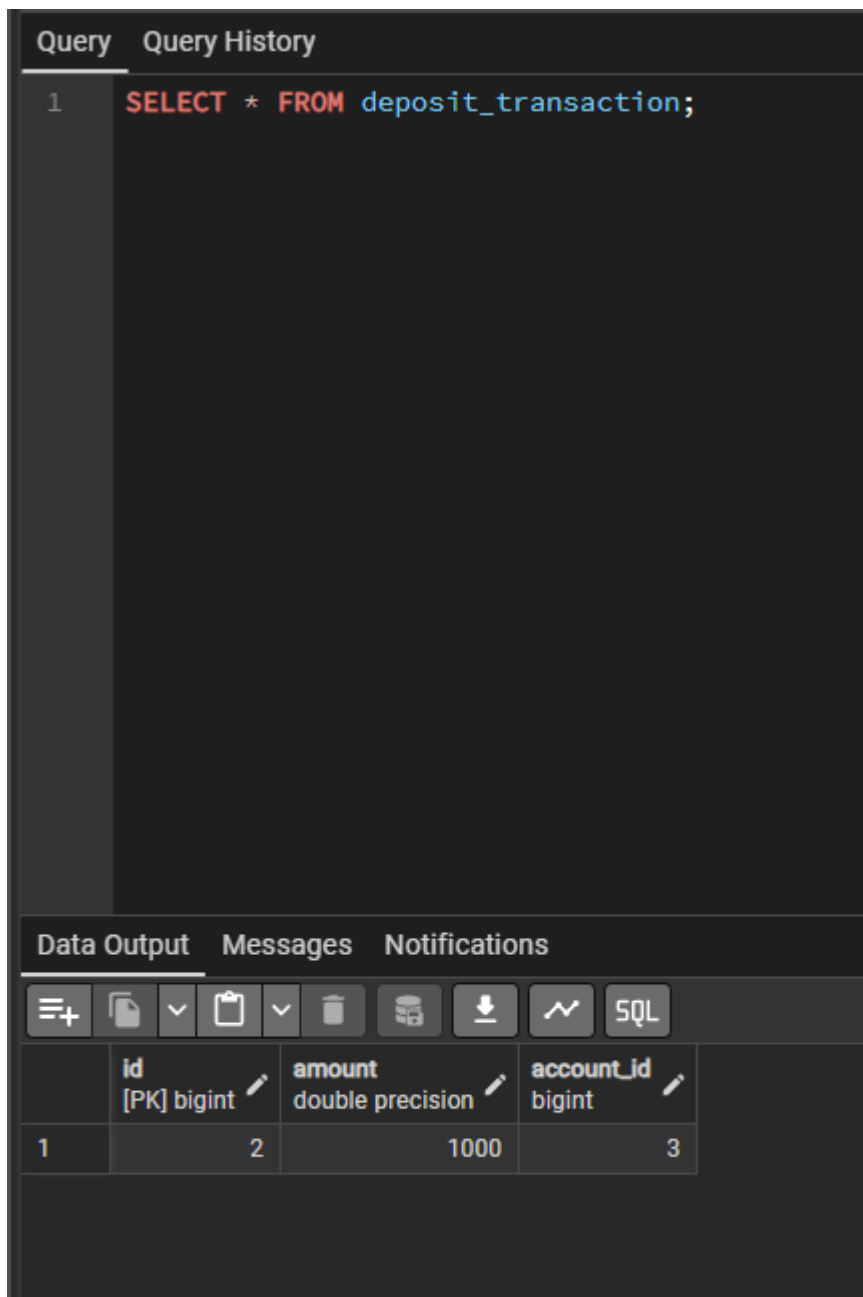


- อธิบายผล

เรียก `GET /accounts/3` เพื่อดูข้อมูลบัญชี พบว่า `balance` เปลี่ยนจาก `0` เป็น `1000` ยืนยันว่าการฝากเงินในข้อ 2 ถูก `COMMIT` ลงฐานข้อมูลจริง ยอดเงินในบัญชีอัปเดตถูกต้อง

4. ตรวจสอบ DepositTransaction

- รูปภาพ

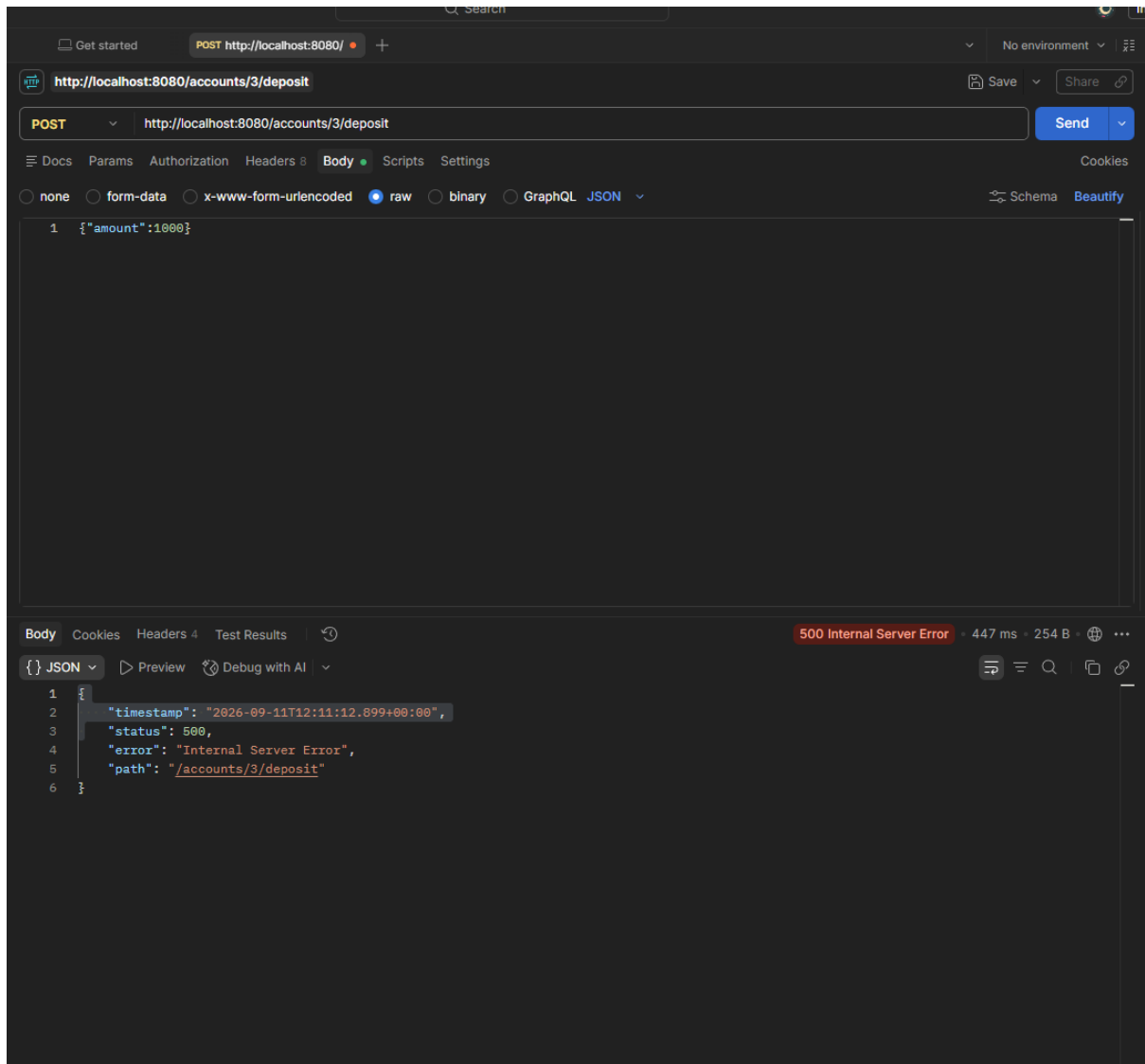


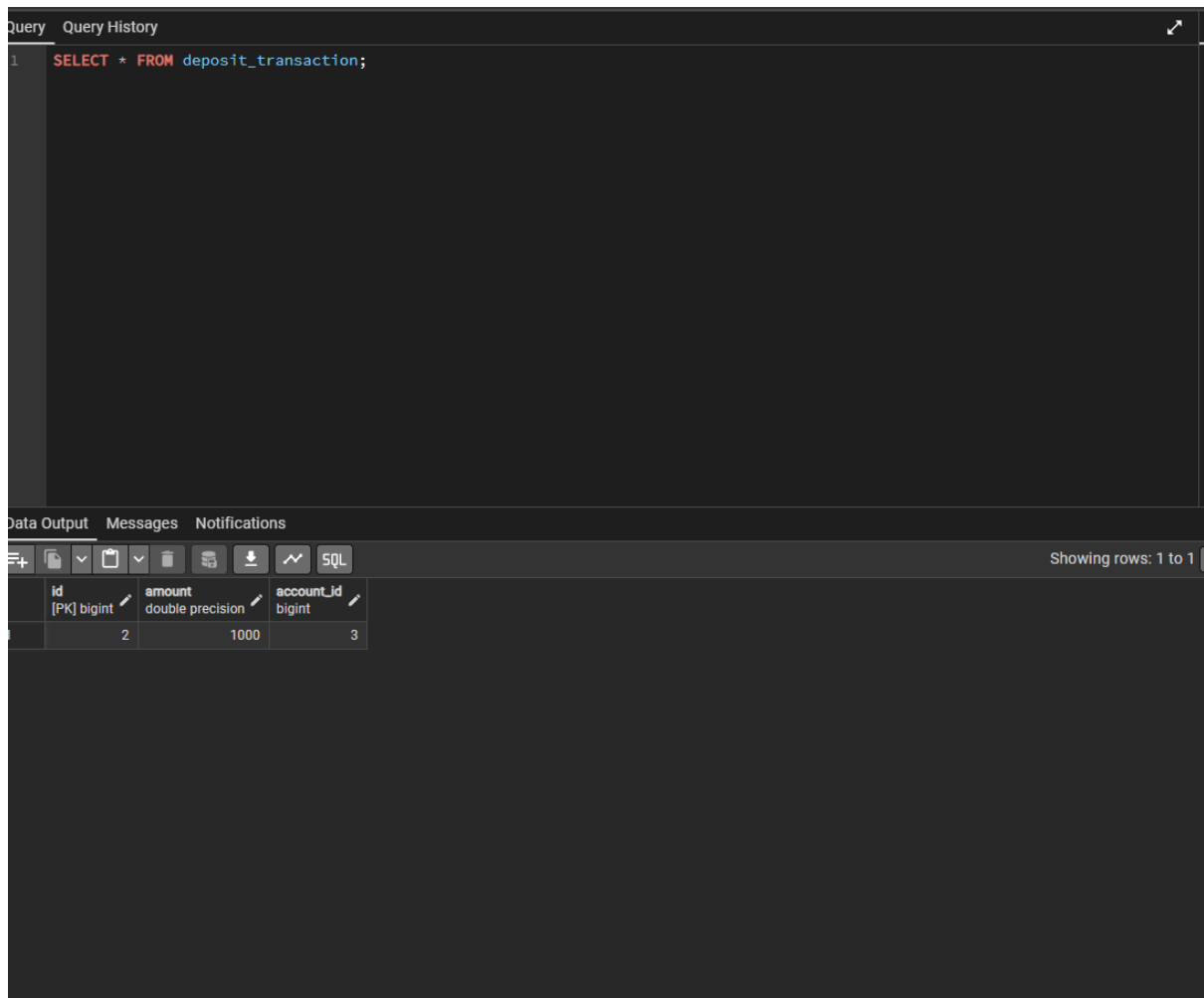
- อธิบายผล

เปิดตาราง `deposit_transaction` ในฐานข้อมูล พบ 1 แถวที่บันทึกจำนวนเงินที่ฝาก (`amount = 1000`) และคอลัมน์ `account_id = 3` ที่ชี้กลับไปยังบัญชีที่ฝาก แสดงให้เห็นความสัมพันธ์แบบ Many-to-One (1 บัญชีมีประวัติการฝากได้หลายรายการ) และยืนยันว่าประวัติการฝากถูกบันทึกคู่กับการเพิ่ม `balance` จริง

5. ทดลอง Rollback

- รูปภาพ





- อธิบายผล

แม้โค้ดจะสั่ง `save()` เพิ่ม `balance` และบันทึกประวัติไปแล้วก่อนเกิด `Error` แต่เพราะมี `@Transactional` เมื่อเมธอดโยน `Exception` ระบบจึงสั่ง `ROLLBACK` ยกเลิกทุกอย่างในทรานแซกชันนั้น ผลคือ `balance` ยังเป็น 1000 เท่าเดิม และตาราง `deposit_transaction` ไม่มีแถวใหม่เพิ่ม พิสูจน์ว่า `@Transactional` ทำงานถ้าขั้นตอนใดพังทั้งก่อนต้องถูกยกเลิก

6. เปรียบเทียบมี `@Transactional` กับไม่มี `@Transactional`

- รูปภาพ

POST http://localhost:8080/accounts/3/deposit Send

Docs Params Authorization Headers 8 **Body** Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☒ JSON Schema Beautify

```
1 {"amount":1000}
```

Body Cookies Headers 4 Test Results 500 Internal Server Error • 33 ms • 254 B 🌐 ⋮

{} JSON Preview Debug with AI 🔍 🔧 🔍 📄 🔗

```
1 path
2 "timestamp": "2026-09-11T12:15:26.111+00:00",
3 "status": 500,
4 "error": "Internal Server Error",
5 "path": "/accounts/3/deposit"
6
```


2. เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

เพราะการฝาก 1 ครั้งเปลี่ยนข้อมูลมากกว่า 1 อย่าง (เพิ่ม balance + บันทึกประวัติ) ทั้งสองต้องสำเร็จพร้อมกัน ถ้าทำสำเร็จอย่างเดียวข้อมูลจะไม่ตรงกัน เช่น เงินเพิ่มแต่ไม่มีประวัติ Transaction จึงคุมให้ "สำเร็จทั้งคู่ หรือไม่สำเร็จเลย"

3. COMMIT และ ROLLBACK ต่างกันอย่างไร?

COMMIT = ยืนยันบันทึกการเปลี่ยนแปลงทั้งหมดลงฐานข้อมูลถาวร / ROLLBACK = ยกเลิกการเปลี่ยนแปลงทั้งหมดในทรานแซกชัน ทำให้ข้อมูลกลับไปเหมือนก่อนเริ่ม

4. Account และ DepositTransaction มีความสัมพันธ์แบบใด?

One-to-Many (1 Account มีได้หลาย DepositTransaction) หรือมองจากฝั่ง DepositTransaction คือ Many-to-One โดยผูกด้วย Foreign Key คอลัมน์ account_id

5. เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

ไม่เปลี่ยนแปลงเลย ระบบ ROLLBACK ยกเลิกทุกคำสั่ง save() ที่ทำไปก่อนหน้านี้ ข้อมูลกลับไปเหมือนก่อนเริ่มทรานแซกชัน

6. เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

ไม่มีการ ROLLBACK คำสั่ง save() แต่ละอันบันทึกลงฐานข้อมูลจบไปทันทีที่คำสั่ง พอเกิด Error ที่หลัง ข้อมูลที่บันทึกไปแล้วจึงค้างอยู่ (balance เพิ่มจริง + มีแถวใหม่) ทำให้ข้อมูลไม่สอดคล้องกัน